

Optimizing Cold-start for Knative Using Predictive Scheduling and Machine Learning

Nguyen Ngoc Tieu Long (250202047)

*Faculty of Information Technology
University of Information Technology*

Supervisor: PhD. Huynh Van Dang

*Faculty of Information Technology
University of Information Technology*

June 20, 2026

Abstract

Serverless computing has gained immense popularity due to its auto-scaling capabilities and pay-as-you-go billing model. However, the cold-start phenomenon remains a major challenge, causing significant latency for initial requests when no active instances are available. This research addresses the cold-start problem in Knative—a Kubernetes-based serverless platform—by proposing a predictive scheduling approach using Machine Learning. Instead of relying on Knative’s default reactive scaling mechanism (Knative Pod Autoscaler), we developed a Long Short-Term Memory (LSTM) time-series forecasting model to predict future workload trends based on historical traffic data collected via Prometheus. Based on these predictions, a proactive “Pre-warming” strategy is implemented to initialize and keep an appropriate number of pods warm before actual traffic surges occur. Experimental results in a simulated environment demonstrate that our proposed predictive system significantly reduces cold-start latency compared to the default mechanism. Furthermore, the study evaluates the trade-off between latency reduction and idle resource consumption, proving that the solution provides an effective and balanced approach for optimizing serverless performance in dynamic traffic environments.

Acknowledgments

First and foremost, I would like to express my sincere gratitude to my supervisor, PhD. Huynh Van Dang, for their invaluable guidance, continuous support, and patience throughout the duration of this project. Their deep knowledge and insightful feedback were essential in helping me navigate the complexities of Kubernetes, Knative, and Machine Learning. I also want to extend my thanks to the professors and staff at the Faculty of Information Technology, University of Information Technology, for providing me with a strong academic foundation and an excellent learning environment over the past years. Finally, I am deeply grateful to my family and friends for their endless encouragement, under-

standing, and emotional support during the challenging phases of my research.

1 Overview

1.1 Problem Statement

Serverless computing architecture is increasingly being widely adopted due to its ability to automatically scale resources (autoscaling) and its pay-as-you-go billing model based on actual usage. However, one of the biggest limitations of serverless is the cold-start phenomenon [7]: when there is no “warm” instance (container/pod) available to process, the system must initialize a new container from scratch, causing significant latency for the first request.

Knative [2] is an open-source serverless platform running on Kubernetes [4], utilizing the Knative Pod Autoscaler (KPA) mechanism to automatically scale the number of pods based on metrics like current concurrency or requests-per-second. This mechanism is reactive—it only scales up when actual traffic arrives—so it cannot completely eliminate cold-start latency, especially in scenarios with sudden traffic fluctuations or for functions with long container initialization times.

1.2 Objectives

The project aims at the following specific objectives:

1. Build a workload forecasting system based on historical data collected from Prometheus [3] to proactively predict near-future traffic trends.
2. Design and implement a Predictive Pre-warming strategy—keeping an appropriate number of pods “warm” before actual traffic increases, instead of waiting and reacting after the traffic has arrived.
3. Integrate the forecasting module (using a Machine Learning model built with PyTorch [9]) into Knative’s autoscaling pipeline.
4. Quantitatively evaluate the effectiveness of the proposed solution compared to the default Knative

Pod Autoscaler mechanism in two aspects: (i) cold-start latency and (ii) idle resource cost.

1.3 Scope of the Project

The project focuses on testing within a small/medium-scale Kubernetes cluster (single-node or multi-node cluster), using simple HTTP functions/services as test subjects. The project does not delve into security issues, multi-tenancy, or optimizations for stateful workloads.

1.4 Summary of Implementation

To achieve the above objectives, the project's implementation includes:

- Deploying Knative on Kubernetes as the serverless operating platform.
- Using Prometheus to collect metrics on traffic and function startup latency.
- Building a Machine Learning model for workload forecasting using Python and PyTorch.
- Designing a proactive Pre-warming strategy based on forecasting results.
- Evaluating and balancing idle resource costs and system response time.

The remainder of the paper is organized as follows: Section 2 presents the foundational technologies and methods used; Section 3 describes the system implementation process in detail; Section 4 presents the test scenarios and analyzes the results; finally, Section 5 summarizes the project and proposes future directions.

2 Related Technologies

2.1 Kubernetes

2.1.1 Concept

Kubernetes (K8s) [4] is an open-source platform used for automating the deployment, scaling, and management of containerized applications.

2.1.2 Characteristics/Capabilities

Describe core components: Pod, Deployment, Service, ReplicaSet, Horizontal Pod Autoscaler (HPA), etcd, control plane/worker node...

2.1.3 Function/Role in the Project

Kubernetes acts as the infrastructure layer for Knative to operate upon.

2.1.4 Why Choose Kubernetes

Reasons for selection: industry standard (de-facto standard) for container orchestration, rich ecosystem, direct compatibility with Knative.

2.2 Knative

2.2.1 Concept

Knative [2] is an open-source framework running on Kubernetes, providing the necessary components to build, deploy, and manage modern serverless applications.

2.2.2 Characteristics/Capabilities

Describe the main components of Knative Serving:

- **Activator:** Receives requests when no pods are running (scale-to-zero), keeping requests in a queue until a pod is ready.
- **Autoscaler (KPA - Knative Pod Autoscaler):** Determines the necessary number of pods based on current concurrency/RPS.
- **Queue-Proxy:** A sidecar container that measures metrics and limits concurrency for each pod.
- **Revision / Route / Service:** Resources (CRDs) that manage versions and route traffic.

2.2.3 Function/Role in the Project

Knative is the main subject of intervention, extending the default autoscaling mechanism with the forecasting module.

2.2.4 Why Choose Knative

Briefly compare with other serverless platforms (OpenFaaS, Kubeless, AWS Lambda) and reasons for selection: open-source, runs on Kubernetes, customizable autoscaler via External Scaler (such as KEDA [1]) or Custom Metrics mechanism.

2.3 Prometheus

2.3.1 Concept

Prometheus [3] is an open-source monitoring system and time-series database.

2.3.2 Characteristics/Capabilities

Pull-based scraping model, PromQL query language, native integration capabilities with Kubernetes and Knative.

2.3.3 Function/Role in the Project

Collecting historical data on request rate, concurrency, and pod count over time—acting as a training data source for the forecasting model.

2.3.4 Why Choose Prometheus

It is the default/recommended monitoring tool in the Kubernetes and Knative ecosystem, easy to integrate, with a large supporting community.

2.4 Machine Learning for Time-Series Forecasting

2.4.1 Concept

Time-series forecasting is the problem of predicting future values of a variable based on previously observed values in the past.

2.4.2 LSTM Model

Long Short-Term Memory (LSTM) [5, 6] is a variant of Recurrent Neural Networks designed to overcome the vanishing gradient problem when learning long-term dependencies in data sequences. It has proven highly effective in forecasting bursty and periodic traffic patterns commonly seen in serverless environments.

The hidden state of an LSTM cell at time t is updated via gates as follows:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (1)$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (2)$$

$$h_t = o_t \odot \tanh(C_t) \quad (3)$$

where f_t is the forget gate, i_t is the input gate, and h_t is the output hidden state.

2.4.3 Why Choose LSTM

Reason: suitable for workload data with cyclical patterns (hourly/daily/weekly), moderate computational complexity compared to Transformer architectures, easy to deploy for real-time inference with low latency.

2.5 Methodology/Algorithm: Predictive Pre-warming

2.5.1 Concept

Predictive Pre-warming is a proactive strategy to maintain a minimum number of pods in a “warm” state based on workload forecasting results, rather than just scaling reactively based on current traffic like the default KPA mechanism.

2.5.2 Formula for determining the number of pods to pre-warm

The minimum number of pods to maintain at time $t + \Delta t$ can be estimated according to the formula:

$$N_{\text{pod}}(t + \Delta t) = \left\lceil \frac{\hat{Y}(t + \Delta t)}{C_{\text{target}}} \right\rceil \quad (4)$$

where $\hat{Y}(t + \Delta t)$ is the workload value (concurrency/RPS) predicted by the LSTM model at time $t + \Delta t$, and C_{target} is the target concurrency threshold per pod (a configuration parameter in Knative). Details on how to integrate formula into the controller are presented in Section 3.5.

2.5.3 (Optional) Deep Reinforcement Learning for Coordination

Introduce the concept of Reinforcement Learning (RL) and its potential role in learning optimal scaling policies. RL agents can effectively balance the latency reduction and resource costs by continuously adapting to traffic changes [8]. The feasibility of this content is evaluated in Section 4.5.

3 Implementation

3.1 Overall System Architecture

Figure ?? (to be added) describes the overall system architecture, including the main components:

1. **Kubernetes Cluster + Knative Serving:** The platform running serverless functions/services.
2. **Prometheus:** Collects real-time metrics from Knative Autoscaler, Activator, Queue-Proxy.
3. **ML Forecasting Service:** The workload forecasting inference service, receiving data from Prometheus, returning predicted values.
4. **Predictive Scaler Controller:** The coordination component, reading forecast results and adjusting Knative’s autoscaling configuration (e.g., via min-Scale or External Scaler/KEDA).

The data flow operates in a cycle: Prometheus collects metrics → ML Service queries historical data and generates forecasts → Predictive Scaler Controller calculates the required number of pods according to formula (2.4) → Controller calls Kubernetes/Knative API to adjust the pod count.

3.2 Environment Installation and Configuration

3.2.1 Installing the Kubernetes Cluster

Describe the tool used (minikube/kind/k3d or cloud K8s) and the installation steps.

3.2.2 Installing Knative Serving

Describe the installation of Knative Serving along with the networking layer (Kourier or Istio).

```
kubectl apply -f https://github.com/knative/serving/releases/download/knative-v1.x/serving-crds.yaml
kubectl apply -f https://github.com/knative/serving/releases/download/knative-v1.x/serving-core.yaml
kubectl apply -f https://github.com/knative-extensions/net-kourier/releases/download/knative-v1.x/kourier.yaml
```

Listing 1: Installing Knative Serving with Kourier

Listing 3.1 illustrates the basic installation steps (need to replace with specific versions when deployed in reality).

3.2.3 Configuring Prometheus

Describe installing Prometheus Operator/ServiceMonitor to scrape metrics from Knative Autoscaler and Queue-Proxy.

3.3 Building the Data Collection and Preprocessing Pipeline

Describe the PromQL queries used to extract request rate/concurrency data over time, and preprocessing steps: resampling at fixed time intervals, normalization, creating sliding windows for the time-series forecasting problem.

3.4 Developing the Machine Learning Model

Present the specific LSTM network architecture (number of layers, hidden state size, input/output window length), the training process, and packaging the model into an inference service (REST API).

3.5 Developing the Predictive Scaler Coordinator

Present the detailed design of the Predictive Scaler Controller: how to read forecast results, apply formula (2.4), call the Kubernetes API to adjust the autoscaling configuration, and a safe fallback mechanism when the forecasting model deviates significantly from reality.

3.6 Load Testing Tools

Describe the load generation tools (k6/Locust/hey) and traffic scenarios designed to simulate different patterns, serving the tests in Section 4.

4 Test Scenarios and Results

4.1 Test Environment

Describe hardware/cloud configuration, Kubernetes/Knative versions, resource parameters (CPU/RAM) allocated for the test cluster.

4.2 Scenario 1: Cold-start Latency Comparison

4.2.1 Specification

Describe the traffic pattern used, number of test iterations, and two comparison configurations: (a) default Knative Pod Autoscaler, (b) the proposed Predictive Pre-warming system.

4.2.2 Results

Results are presented in Table ?? (actual data to be added).

4.2.3 Discussion/Evaluation

Analyze the latency reduction and the conditions under which the proposed system operates effectively or ineffectively.

4.3 Scenario 2: Evaluating Forecasting Model Accuracy

4.3.1 Specification

Describe the test dataset and evaluation metrics: MAE, RMSE, MAPE.

4.3.2 Results

Present charts comparing forecasted values with actual values over time.

4.3.3 Discussion/Evaluation

Analyze the prediction lag between forecasts and actual events, and the impact of the prediction horizon length on accuracy.

4.4 Scenario 3: Evaluating Resource Costs

4.4.1 Specification

Describe the measurement methodology: pod-hour, CPU/memory consumption in each scaling strategy.

4.4.2 Results

Comparison table of estimated costs between configurations.

4.4.3 Discussion/Evaluation

Analyze the trade-off between latency reduction and increased idle resource costs; propose an optimal balance point (appropriate pre-warming threshold).

4.5 Scenario 4: Stress Test with Abnormal Traffic

4.5.1 Specification

Describe a sudden traffic scenario that does not follow learned patterns to test system robustness.

4.5.2 Results

Present measurement results.

4.5.3 Discussion/Evaluation

Evaluate the system's resilience when the forecasting model encounters out-of-distribution data.

4.6 Scenario 5 (Optional): Feasibility of Deep Reinforcement Learning

4.6.1 Specification

Compare the scaling policy learned by DRL with the heuristic based on LSTM forecasts.

4.6.2 Results

Present comparison results.

4.6.3 Discussion/Evaluation

Analyze the pros, cons, and practical implementation complexity of the DRL approach.

5 Conclusion

5.1 Summary

Summarize the achieved results, mapping them to each objective set out in Section 1.2:

- Cold-start latency reduction achieved (citing data from Section 4.2).
- Workload forecasting model accuracy (Section 4.3).
- Resource cost efficiency (Section 4.4).

5.2 Limitations

- The training data used in testing is still simulated and does not fully reflect the diversity of real-world traffic.
- The scale of the test cluster is small, not yet evaluating scalability at a production scale.
- The Machine Learning model needs to be retrained periodically to adapt to changing traffic patterns over time.

5.3 Future Directions

- Expand the architecture towards multi-model serving, supporting multiple types of functions/services with different workload characteristics.
- Apply online learning mechanisms for the model to update automatically with new data, reducing the need for manual retraining.
- Fully develop the Deep Reinforcement Learning approach evaluated for feasibility in Section 4.6.
- Test on a real cloud environment with a larger scale and more diverse workloads.

References

- [1] KEDA Authors. Keda: Kubernetes event-driven autoscaling. <https://keda.sh/>, 2024. Last accessed: 2026.
- [2] Knative Authors. Knative documentation. <https://knative.dev/docs/>, 2024. Last accessed: 2026.
- [3] Prometheus Authors. Prometheus: Monitoring system & time series database. <https://prometheus.io/docs/introduction/overview/>, 2024.
- [4] The Kubernetes Authors. Kubernetes documentation. <https://kubernetes.io/docs/home/>, 2024. Last accessed: 2026.
- [5] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [6] T. Nguyen et al. Time series forecasting for serverless cold-start mitigation using lstm. *IEEE Cloud Computing*, 2023.
- [7] Mohammad Shahrad, Rodrigo Fonseca, Iñigo Goiri, Gohar Chaudhry, Paul Batum, Jason Cooke, Eduardo Laureano, Colby Tresness, Mark Russinovich, and Ricardo Bianchini. Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, pages 205–218, 2020.
- [8] Amog Suresh et al. Pre-warming serverless functions with reinforcement learning. *arXiv preprint arXiv:2002.00000*, 2020.
- [9] PyTorch Team. Pytorch documentation. <https://pytorch.org/docs/stable/index.html>, 2024.

A Appendix

Full source code, configuration files, etc. can be placed here or linked to a repository.